

Destiny 2 Activity System & Authored-Content Internals

How an activity becomes a real mission: the identity-1 route/mode pipeline, the selection descriptors, the authored-launch manager, its full offline dormancy chain, and the server publication side - with functions and addresses.

A technical reference for the Sunrise offline-revival project. All addresses are for the pinned Season-of-Arrivals image, base 0x7FF618070000 (file offset == RVA; VA = base + RVA). Reconstructed from reverse engineering of the local client, the Sunrise embedded server, and collaborator George Ratington's map (verified to match this build).

Revision 2026-08-18

1. Overview - what makes a mission a mission

Destiny 2's networking splits into a control side (a Bungie service bus this project calls **BAP**, riding on the Demonware middleware in passing) and a peer-to-peer game session. But the thing that decides whether `mission_towerfall` is a live mission or an empty shell is not the transport - it is the **activity selection / route system** on the client. This document is about that system: how a selection descriptor drives an *identity's* activity manager into **authored 'mode 1'** (which builds the mission director, activity script, encounters, AI, cinematics) versus local **'mode 6'** (which builds none of it).

The single crux, stated once: **everything gates on identity 1 (the mission identity) entering authored mode 1 instead of local mode 6**. Offline, it never does - the whole authored-content pipeline stays dormant - and the rest of this reference is the anatomy of why, function by function.

```
identity 0 = orbit          identity 1 = mission          (manager+0x854 = identity index)
mode 6  = LOCAL manager  -> skips authored receiver/component creation  <-- always, offline
mode 1  = AUTHORED manager -> builds receiver/component/dispatch:
                        mission director (slot 35), activity script (slot 18), encounters, AI, cinematics
```

Figure 1. The mode-6-vs-mode-1 crux. Local vs authored is chosen per identity slot by a single route/mode byte the offline path never sets.

2. The activity managers

Two manager class families exist; which one is constructed for an identity is the whole game:

Element	VA / RVA	Role
local manager type	+0x1772440	mode-6 local manager; skips authored receiver creation
authored manager type	+0x1773200	mode-1 authored manager; builds the content pipeline
receiver/component path	+0x17B8C50	authored: creates receivers + components
component dispatch	+0x1766A30	authored: dispatches components (script/director/encounters)
local mgr init	FUN_7FF6197E2440	shared init; dispatcher passes mode 4 (local) / 0xC (authored)
authored mgr init	FUN_7FF6197E3200	mode-1 authored init
identity index	manager+0x854	0 = orbit, 1 = mission
activity-client slot	manager+0x27D0+identity*0x2D0	unset

3. The client-side authored pipeline

When a real selection is ingested, this chain runs. Every link is verified against runtime measurement; offline it runs with an empty/local record and produces mode 6.

```
BAP server publishes an activity selection (activity MESSAGE 1 / global-state push)
-> client ingests; native producer builds the 0x1B0 RUNTIME selection descriptor
-> descriptor ROUTE BYTE at +0x12  (0 = local, nonzero = authored)
-> route_commit FUN_7FF6197C1F50  branches on +0x12
-> route_update FUN_7FF61981C390  applies the route to the manager
-> selection pump FUN_7FF6197CE520  reads per-slot state[slot*0x1B + 2]:
    == 0  -> LOCAL  init FUN_7FF6197E23C0 -> FUN_7FF6197E2440  (mode 6)  <-- always offline
    != 0  -> AUTHORED init FUN_7FF6197FFE00 / FUN_7FF6197E1AA0 -> FUN_7FF6197E3200 (mode 1)
-> authored manager builds receiver/component (FUN_7FF6197B8C50) + dispatch (FUN_7FF61766A30)
```

Figure 2. The route/mode decision chain. The pump's per-slot +2 byte is the actual local-vs-authored switch; for identity 1 it is 0 offline.

3.1 Where the route byte comes from - the root selection holder

The route byte is never written by anything downstream; it is carried by a selection record that propagates from a single root holder. The holder was resolved from a live vtable pointer and both setter slots detoured:

Holder element	RVA	Identity
holder class vtable	0x1CA79D0	the selection-holder vtable
get (vtable +0x98)	0x1778C70	returns holder+0x148 - the record slot
set (vtable +0xB0)	0x17AD310	writes the PER-SLOT holder ctx(slot)+0x19068
set (vtable +0xB8)	0x17AB350	writes the ROOT holder ctx(0)+0x18DD0
fetch+gate	FUN_7FF6197C51A0	141 bytes: if rec[0]==5 && rec[0xA1]==0 -> route_commit
reader	FUN_7FF6197C1370	reads ctx+0x19068 via vtable +0x98

Measured: the root holder $\text{ctx}(0) + 0x18DD0$ is **never populated offline** (the +0xB8 setter recorded zero calls across a full session). The only writer is the pump itself, which writes **local** records (route=0, source=1, dest=-1, kind cycling 1,2,3,5,6,7,0) into the per-slot holder. Slots 1 and 2 never receive a record. So the route is local for want of an authored record, not because a value is wrong.

3.2 The record contract (0xA8 bytes)

The consumers accept a record only when:

- $+0x00 \text{ kind} == 5$ $+0xA1 == 0$ $+0x18 \text{ id}$ not 0 or 1
- **route byte** $+0x12 \text{ nonzero}$ = authored $+0x08 \text{ source}$ $+0x0C \text{ dest}$ $\text{payload} + 0x20 \dots 0x9F$ $+0xA0 \text{ kind word}$

4. The two selection descriptors

4.1 The 0x1B0 runtime descriptor

Built by the client's native producer, lives at accessor (FUN_7FF618C69FA0)+0xA30, published by FUN_7FF61981DA60 (RVA 0x17ADA60, the route chokepoint). dword0 is packed $\text{reason}[0:4]$ $\text{source}[4:16]$ $\text{dest}[16:28]$; route byte at +0x12; package name near +0x50. The minimal/local form (all we ever see offline) has small source/dest and a body of FFFF0000 sentinels - George's '372-bit minimal, stays local mode 6'. The authored form is only built when a real CHOSEN selection is ingested, which never happens offline.

4.2 The wire descriptor (BAP selection) and the opaque delta

Parser: `activity_manager_descriptor_parser.cpp`. From bit 0, each field wire-biased by +1:

```
reason[4] source[12] dest[12]
optional element index (presence + 9 bits)
skull count[5] then skullCount x (group[2] + value[7])
unknown byte[8] optional peer byte
optional arrival-bubble hash[32] optional spawn-set hash[32]
package name (40 signed bytes, bias 128) trailing bool[1]
tail array 1 (1-bit count, 96-bit 'reference' entries) tail array 2 (6-bit count, 13-bit entries)
```

Minimal form = 372 bits exactly (matches the code's own comment). Authored form is roughly 620 bits - a ~248-bit delta. That delta was decoded from real offline captures (via Sunrise's svc-6 parse instrumentation): it is **our own SOID, three FNV 'none' sentinels, sentinels, a peer byte, two tail arrays, and one 64-bit runtime nonce**:

Delta field	Value (measured)	Sourceable offline?
sensitive scalar #1	0x9EAA300100100100 = our primary_soid	YES - from settings.json
sensitive scalar #2	the fireteam/activity nonce (re-minted per setup)	Capturable, but timing (below)
arrival/spawn/unknown has	0x811C9DC5 (FNV-1a 'none')	trivially
element index	-1 (absent sentinel)	trivially
refs tail (96-bit)	count 0 or 1 (per-activity)	the hard unknown when present

The hard ordering problem: across three measured loads the nonce is minted *after* identity-1's manager is already built local and its route already committed (by 15-94 ms). So a descriptor for load N cannot be assembled in time for load N - its nonce does not yet exist. A replay necessarily uses a stale nonce from load N-1, and whether the client accepts a foreign nonce is untested. This is why message-1 injection (proven delivered, correct timing, real 620-bit body) still produced local route.

5. The authored-launch manager and its dormancy chain

Upstream of the pump sits the **authored-launch manager** - the subsystem that would stamp a record with route=AUTHORED. Offline it is entirely dormant. Every stage was forced or bypassed in turn to expose the next; this is the full chain, top to bottom:

Stage	Function / field	RVA	Offline state
1. activate	FUN_7FF618F87360 sets mgr+0x81=1	0xF17360	never runs (vtable-dispatched)
2. populate	mgr+0x08/+0x48/+0x148 payload	-	never set (message/event-driven)
3. arm decision	FUN_7FF618F94CF0 (periodic tick)	0xF24CF0	runs; declines
4. arm action	FUN_7FF618F907B0 -> setter F971C0	0xF207B0	never fires
5. staged flag	obj+0x3F0 / payload obj+0x3D0	-	never armed (reset by tick)
6. consumer tick	FUN_7FF618F5E750 (gates 1-5)	0xEE750	ticks; gate1 pending never set
7. ctor caller	FUN_7FF618F93A50 (mgr+0x8c / +0x13c)	0xF23A50	never reached
8. authored ctor	FUN_7FF6197C7C00 (stamps route=AUTHORED)	0x1757C00	the target

5.1 The arm decision (why it declines)

The arm tick FUN_7FF618F94CF0(mgr) arms only when all hold: mgr+0x81!=0 (active), activity slot 1 exists, mgr+0x148!=0, mgr+0x08!=0 (payload), mgr+0x48!=0, mgr+0x141==0, and the two activity slots **NOT both committed**. Offline both slots commit LOCAL and stay committed the whole window, so the tick always takes the both-committed branch and returns without arming - independent of the payload fields. The activator FUN_7FF618F87360 (sets mgr+0x81=1, mgr+0x84=armval) and the tick are both vtable-dispatched with zero static xrefs - virtual methods on the manager class, driven by a higher state machine.

5.2 The selection dispatcher and the fork

At the bottom, the selection dispatcher FUN_7FF6193C0170 (RVA 0x13C0170) branches on param_6 (sourced from manager+0x148 via caller FUN_7FF6193C17B0 / RVA 0x13C17B0):

- param_6 != 1 -> FUN_7FF6197CB8F0 = fireteam-join (a dead end for missions - it queues a session join to a null peer and hangs on 'JOINING FIRETEAM').
- param_6 == 1 -> FUN_7FF6197C7C00 = the **authored-route constructor**: memsets a 0xA8 record, fills +0x18 from the id generator FUN_7FF61844DB90, copies the 0x80 payload, bakes in route/kind const

0x00010204 -> route byte = AUTHORED. Its payload block is lVar5+0xbc (empty offline).

5.3 The trigger gate and the spoof that fired the ctor

The whole subsystem waits on one gate: FUN_7FF6197D3B20 (RVA 0x1763B20) calls the authored launch dispatcher FUN_7FF6193BDF0 (RVA 0x134DF0, writes manager+0x148) only when all of:

- DAT_7FF61B24C431 == 0 (a suppressed-latch; measured = 1 always = the master blocker)
- FUN_618E9D510() != -1 and == FUN_618E9D440(0) (a session PHASE == lane-0 state; measured: matched naturally at phase 29)
- FUN_61853DEB0(0) != 0 (a 'ready' byte; measured = 0 always)

A one-shot spoof (set DAT_7FF61B24C431=0 + make the ready byte nonzero around a single native gate call, then restore) made the real dispatcher run offline for the first time; adding a fork-force (param_6=1) then drove it to the authored ctor.

5.4 Milestone and where it stalls

The authored constructor FUN_7FF6197C7C00 **fired offline for the first time** (gate spoof + fork force), no crash - the deepest reach. But it fired with **empty data** (payload all zeros), so the authored launch immediately performed an outgoing session join with a **null target** and hung on 'Waiting for pending fireteam leave to complete'. Confirming the architecture: **an authored mission launch is itself a session/fireteam join**. With a filled payload the join targeted a specific *player* ('unable to join target player') - the wrong model. This is what pivoted the effort to the public/peer path (below).

6. The destination predicate

FUN_7FF618C768F0 (RVA 0xC068F0, (short activityIndex, char* name)) validates whether the current destination datum matches a requested activity. It looks up the destination record via FUN_7FF618439450(idx) (datum lookup (idx&0x1FFF)*size+base), checks readiness (FUN_7FF61857BF00), and compares the destination name to the requested activity (by name FUN_7FF61843C1E0, or by index). Its gate FUN_7FF618C91370 fails when the activity-client slot manager+0x27D0+identity*0x2D0 == -1. **Runtime probe result: it ACCEPTS 266** (returns 0x1 when asked 'is destination 266?', destination datum = 0x80B500BC mission_towerfall.scenario). The earlier 'predicate rejects 282->266' was a misdiagnosis; the real block is ordering - the manager commits local before the authored destination exists.

7. Two paths: authored (content) vs public/peer (get-in)

Because the authored fork routes to a fireteam-join that hangs, and authored mode-1 offline is blocked by the per-session nonce timing, mission revival takes a **second, orthogonal path**:

- **Authored path (content)**: everything above. Drives real mission content but is dormant offline and, when forced, hangs on the null session join. Currently observe-only.
- **Public/peer path (get-in)**: force the mission's slice set 'public' (client reader FUN_7FF618C910F0 / RVA 0xC210F0, at the transition starter +E2B3BD) so the client takes the patrol-style **citizen join** and connects to the embedded host as a peer. This establishes a real session + entity slots and loads the world. It is **mutually exclusive** with the authored spoof for a mission (the authored fork bypasses the public transition).

The two must ultimately converge: the public path gets a live-but-empty instance; the authored layer is what fills it. Reaching a peer session is the 'get in' milestone; the authored content is the endgame on top of it.

8. The server publication side

Sunrise's embedded server is where an authored selection would be published. What is built:

- `server/bap/encrypted/push/activity/activity_global_state_push.cpp` - the message-1 global-state push (sends `selection.descriptorBits`). The route lever, earlier than service 6.
- `server/bap/encrypted/activity_host_manager/activity_host_manager_route.cpp` - service 6; **echoes** the client's descriptor bits, does not build them; fires AFTER the route commits, so it is not the route lever.
- `state/activity/forced/activity_forced_destination.cpp` - the Activity Override; synthesizes a full 620-bit authored descriptor when `is_towerfall` (`build_authored_descriptor`, real SOID + host nonce) and logs `ev=bap stage=authored result=built source=282 dest=266`.
Production is done server-side; delivery is proven. The lever (what makes the client's runtime descriptor adopt it before the local commit) is the open problem.

So the server can already synthesize and push the authored descriptor; the wall is the client consuming it in time - the dormancy chain of section 5 - not the data.

9. Authored content tags (definition inputs)

The authored content records are present in local packages and read correctly via `middleware/content/packages/reader.h :: read_tag`. These are DEFINITION inputs, not the runtime 0x1B0 descriptor:

Tag	Class / size	Role
0x80FDB97F	0x80809BA3 / 0x30	launch descriptor; children 0x80FEB810 / 0x80FDB980 / 0x80FDB981
0x80B500AC	0x80808AAE / 0x48	mission_towerfall.activity; +0x08 = 0x9ACCB518 (activity hash), +0x40 scenario, +0x44 launch
0x80B500BC	0x80809994 / 0x7D8	mission_towerfall.scenario
0x80B508A9	0x80809C36 / 0xBE2	scenario_patch

10. Master address reference

All at base 0x7FF618070000 (VA = base + RVA).

Function / datum	RVA	Role
<code>FUN_7FF6197CE520</code>	0x175E520	selection pump (per-slot local/authored decision)
<code>FUN_7FF6197C1F50</code>	-	route_commit (branches on +0x12)
<code>FUN_7FF61981C390</code>	-	route_update (applies route to manager)
<code>FUN_7FF6197C51A0</code>	-	fetch+gate to route_commit (kind==5, +0xA1==0)
<code>FUN_7FF61981DA60</code>	0x17ADA60	current-selection publisher (route chokepoint)
<code>FUN_7FF618C6A5A0</code>	0xBFA5A0	activity-selection launcher (copies 0x1B0 from +0xA30)
<code>FUN_7FF618C69FA0</code>	-	selection-launch accessor (+0xA30 = the 0x1B0 desc)
<code>FUN_7FF618547940</code>	-	0x1B0 descriptor init (state-0 producer)
<code>FUN_7FF6197E2440</code>	-	local mgr init (mode 6)
<code>FUN_7FF6197E3200</code>	-	authored mgr init (mode 1)
<code>FUN_7FF6193C0170</code>	0x13C0170	selection dispatcher (param_6 fork)

Function / datum	RVA	Role
FUN_7FF6197C7C00	0x1757C00	authored-route constructor (route=AUTHORED)
FUN_7FF6193BFD0	0x134FDF0	authored launch dispatcher (writes mgr+0x148)
FUN_7FF6197D3B20	0x1763B20	the trigger gate (3 conditions)
FUN_7FF618F94CF0	0xF24CF0	arm-decision tick
FUN_7FF618F907B0	0xF207B0	arm action (stages obj+0x3F0/0x3D0)
FUN_7FF618F87360	0xF17360	manager activator (mgr+0x81=1)
FUN_7FF618F5E750	0xEEE750	consumer tick (gates 1-5)
FUN_7FF618F93A50	0xF23A50	ctor caller (mgr+0x8c / +0x13c gates)
FUN_7FF618C768F0	0xC068F0	destination predicate (accepts 266)
FUN_7FF618C910F0	0xC210F0	slice-set public reader (region_force_public target)
FUN_7FF61844DB90	0x3DDB90	id generator (record +0x18)
read_bits	0x3513B0	bit-stream reader (MSB-first)

11. Key codes, the ceiling, and a note on Demonware

Codes: 282 / 0x11A = CHOSEN source; 266 / 0x10A = mission_towerfall (Homecoming); 48 / 0x30 = Homecoming slice (PUB48.48); 0x9ACCB518 = activity-definition hash; route field +0x12 (0 local / nonzero authored); identity index manager+0x854 (0 orbit / 1 mission); activity-client slot manager+0x27D0+identity*0x2D0 (-1 unset).

The honest ceiling: per George's notes, even his complete pipeline - including a working 282->266 rewrite - still yields no cinematic, objectives, dialogue, AI, encounters, markers, or progression. Reaching authored mode 1 reproduces 'reaches the mission transition, world loads', not a playable mission; the authored content beyond that is server-authoritative and yields nothing offline. Reaching a peer session (the public path) is the achievable 'get in' milestone.

Demonware, in passing: the transport under BAP is the Demonware middleware (bdSocket / bdConnection / bdDTLS / bdNAT / bdQoS). Its QoS, NAT, and DTLS layers are self-contained and peer-to-peer; once the client holds a valid host descriptor they need no dedicated server. It is the plumbing beneath the activity system described here, not the subject of it.

Addresses and field offsets are from reverse engineering of the local pinned image for personal, offline preservation. The authored pipeline is verified against runtime measurement; the authored-launch dormancy chain (section 5) was mapped by forcing each dormant stage in turn.